

STL

ЛАБОРАТОРНАЯ РАБОТА №2

Решение всех задач оформить в виде функций, решающими поставленные задачи. В главной функции main организовать вызов всех функций с дружественным интерфейсом. Если исходные данные вводятся с клавиатуры, то организовать проверку на ввод.

В **заданиях 1, 2, 3** элементами контейнеров являются целые числа. Для заполнения контейнера использовать итератор и конструктор соответствующего контейнера, для вывода элементов использовать итератор (для вывода элементов в обратном порядке использовать обратные итераторы, возвращаемые функциями-членами rbegin и rend).

В **задании 4** обработка данных выполняется без использования контейнеров. Если алгоритм требует применения функционального объекта, следует использовать лямбда-выражения. Использовать итераторы istream_iterator и ostream_iterator.

Если в **заданиях 5, 6, 7, 8, 9, 10** тип элементов контейнера не указан, то предполагается, что элементами являются целые числа.

Необходимо решить по 1 задаче из каждого задания согласно вашему варианту. Задания 1-7 оцениваются по 0,5 балла, задания 8-10 по 1,5 балла. Максимально за лабораторную работу можно получить 10 баллов (8 баллов за решение задач + 2 балла за оформление отчета).

Распределение вариантов

№варианта	№1	№2	№3	№4	№5	№6	№7	№8	№9	№10
1	3	3	10	5	1	1	11	2	1	1
2	3	4	7	1	2	2	12	1	1	2
3	1	8	6	9	3	3	13	2	2	3
4	5	2	8	1	4	4	1	1	2	4
5	6	5	11	5	5	5	2	2	1	1
6	4	7	1	7	6	6	3	1	1	2
7	2	6	2	8	7	7	4	2	2	3
8	6	3	9	4	8	8	5	1	2	4
9	1	6	8	2	9	9	6	2	1	1
10	7	2	10	2	1	10	7	1	1	2
11	3	7	5	7	2	11	8	2	2	3
12	7	1	5	3	3	12	9	1	2	4
13	4	5	3	6	4	13	10	2	1	1
14	6	4	9	3	5	14	11	1	1	2
15	5	8	4	6	6	15	12	2	2	3
16	2	1	11	4	7	1	13	1	2	4

Задание 1. Заполнение и доступ к элементам. Обратные итераторы.

1	Дан текстовый файл с именем name, содержащий строковые представления целых чисел. Заполнить вектор V числами из исходного файла и вывести элементы вектора в исходном порядке.
2	Дан набор целых чисел. Заполнить список L исходными числами и вывести элементы списка L в начале в исходном, а затем в обратном порядке.
3	Дан набор целых чисел с четным количеством элементов. Заполнить вектор V исходными числами, вывести вначале вторую половину элементов вектора V, а затем первую половину (в каждой половине порядок элементов не изменять).
4	Дан набор целых чисел с четным количеством элементов. Заполнить дек D исходными числами, вывести первую половину элементов дека D в обратном порядке, а затем — вторую половину (также в обратном порядке).
5	Дан набор целых чисел, количество которых делится на 3. Заполнить список L исходными числами, вывести вначале первую треть элементов списка L в исходном порядке, затем вторую треть элементов в обратном порядке, а затем последнюю треть (также в обратном порядке).

6	Даны вектор V , дек D и список L . Каждый исходный контейнер содержит не менее трех элементов, количество элементов является нечетным. Удвоить значения первого, среднего и последнего элемента каждого из исходных контейнеров.
7	Даны вектор V , дек D и список L . Каждый исходный контейнер содержит не менее двух элементов, количество элементов является четным. Поменять значения двух средних элементов каждого из исходных контейнеров. Использовать алгоритм <code>swap</code> (не путать его с одноименной функцией-членом контейнера).

Задание 2. Вставка элементов

1	Дан вектор V с четным количеством элементов. Добавить в середину вектора 5 нулевых элементов. Использовать один вызов функции-члена <code>insert</code> .
2	Дан дек D с нечетным количеством элементов $N(\geq 5)$. Добавить в начало дека пять его средних элементов в исходном порядке. Использовать один вызов функции-члена <code>insert</code> .
3	Дан список L , количество элементов которого делится на 3. Добавить в конец списка первую треть его исходных элементов в обратном порядке. Использовать один вызов функции-члена <code>insert</code> .
4	Даны вектор V и список L . Каждый исходный контейнер содержит не менее 5 элементов. Вставить после элемента списка с порядковым номером 5 первые 5 элементов вектора в обратном порядке. Использовать один вызов функции-члена <code>insert</code> .
5	Даны дек D и список L . Каждый исходный контейнер содержит не менее 5 элементов. Вставить перед пятым с конца элементом списка последние 5 элементов дека в обратном порядке. Использовать один вызов функции-члена <code>insert</code> .
6	Даны вектор V и дек D , имеющие четное количество элементов. Добавить в конец вектора первую половину элементов дека (в исходном порядке), а в начало дека — вторую половину исходных элементов вектора (в обратном порядке). Использовать два вызова функции-члена <code>insert</code> .
7	Дан вектор V . Вставить после каждого элемента исходного вектора число -1 . Использовать функцию-член <code>insert</code> в цикле с параметром-итератором. Указание. Организуйте перебор элементов вектора в цикле с параметром-итератором i . Вставку выполняйте в позицию <code>++i</code> , обязательно присваивая параметру i значение, возвращаемое функцией-членом <code>insert</code> .
8	Дан дек D с четным количеством элементов. Вставить перед каждым элементом из первой половины исходного дека число -1 . Использовать функцию-член <code>insert</code> в цикле с числовым параметром. Указание. Используйте цикл с числовым параметром, повторяющийся $N/2$ раз (где N — исходный размер дека). Свяжите вспомогательный итератор i с началом второй половины элементов дека. В цикле выполняйте функцию-член <code>insert</code> для первого параметра, равного <code>--i</code> , обязательно присваивая возвращаемое значение итератору i .

Задание 3. Удаление элементов

1	Дан дек D с нечетным количеством элементов $N(\geq 3)$. Удалить средний элемент дека. Использовать функцию-член <code>erase</code> .
2	Дан вектор V с нечетным количеством элементов $N(\geq 5)$. Удалить три средних элемента вектора. Использовать один вызов функции-члена <code>erase</code> .
3	Даны список L и вектор V ; список L имеет нечетное количество элементов. Переместить средний элемент списка L в конец вектора V . Использовать функции-члены <code>push_back</code> и <code>erase</code> .
4	Даны список L и дек D ; дек D имеет четное количество элементов. Переместить первую половину элементов дека D в начало списка L . Использовать один вызов функций-членов <code>insert</code> и <code>erase</code> .
5	Даны списки $L1$ и $L2$; список $L1$ имеет нечетное количество элементов. Переместить средний элемент списка $L1$ в конец списка $L2$. Использовать один вызов функции-члена <code>splice</code> .
6	Даны списки $L1$ и $L2$; список $L2$ имеет четное количество элементов. Переместить первую половину элементов списка $L2$ в начало списка $L1$. Использовать один вызов функции-члена <code>splice</code> .
7	Даны списки $L1$ и $L2$, имеющие четное количество элементов. Поменять местами первую половину исходного списка $L1$ и вторую половину исходного списка $L2$. Использовать два вызова функции-члена <code>splice</code> .

8	Дан вектор V. Удалить все элементы исходного вектора с нечетными порядковыми номерами (считая, что начальный элемент вектора имеет порядковый номер 1). Использовать функцию-член erase в цикле с параметром-итератором. Указание. Организуйте перебор элементов вектора в цикле с параметром-итератором i, увеличивая параметр в заголовке цикла. Удаление выполняйте в позиции i, обязательно присваивая параметру i значение, возвращаемое функцией-членом erase. В конце каждой итерации цикла дополнительно проверяйте, достигнут ли конец вектора.
9	Дан дек D с количеством элементов, кратным 4. Удалить в первой половине исходного дека все элементы с четными порядковыми номерами (считая, что начальный элемент дека имеет порядковый номер 1). Использовать функцию-член erase в цикле с числовым параметром. Указание. Используйте цикл с числовым параметром, повторяющийся N/4 раз (где N — исходный размер дека). Свяжите вспомогательный итератор i с началом дека. В цикле выполняйте функцию-член erase с параметром, равным ++i, обязательно присваивая возвращаемое значение итератору i.
10	Дан список L с элементами A1, A2, A3, ..., AN-1, AN (N — четное). Изменить порядок элементов в списке на следующий: AN, A1, AN-1, A2, AN-2, ..., AN/2, AN/2-1. Для этого использовать два итератора i и r, связав их с первым и последним элементом списка. В цикле, который должен повторяться N/2 раз, вызывать функцию-член splice с первым параметром i++ и третьим параметром r--.
11	Даны два списка L1 и L2 с одинаковым количеством элементов N. Получить в списке L2 комбинированный набор элементов исходных списков вида A1, B1, A2, B2, A3, B3, ..., AN, BN, где Ai обозначают элементы исходного списка L1, а Bi — элементы исходного списка L2. Для этого использовать итераторы i1 (для списка L1) и i2 (для списка L2), связав их с первым элементом соответствующего списка. В цикле, который должен повторяться N раз, вызывать функцию-член splice для списка L2 с первым параметром ++i2 и третьим параметром i1++.

Задание 4. Итераторы и алгоритмы.

1	Дано целое число K (> 0), текстовый файл с именем name1, содержащий английские слова, и строка name2. Записать в текстовый файл с именем name2 все слова из исходного файла, длина которых не превосходит K, сохранив исходный порядок их следования и располагая каждое слово на новой строке. Использовать алгоритм remove_copy_if.
2	Дан текстовый файл с именем name, содержащий строковые представления целых чисел. Вывести числа из исходного файла с нечетными порядковыми номерами (т. е. первое число, третье число и т. д.). Использовать алгоритм remove_copy_if.
3	Дан набор вещественных чисел, содержащий не менее двух элементов. Вывести числа из исходного набора с четными порядковыми номерами (т. е. второе число, четвертое число и т. д.). Использовать алгоритм remove_copy_if.
4	Дана строка name и набор целых чисел. Записать в текстовый файл с именем name все числа из исходного набора в том же порядке, заменяя каждое число 0 на число 10 и добавляя после каждого числа два пробела. Использовать алгоритм replace_copy.
5	Дан набор символов. Вывести все символы из исходного набора в том же порядке, заменяя цифровые символы на символ подчеркивания. Использовать алгоритм replace_copy_if.
6	Дана строка name и набор символов. Записать в текстовый файл с именем name удвоенные кодовые значения всех символов из исходного набора в том же порядке, добавляя после каждого числа один пробел. Использовать алгоритм transform.
7	Дана строка name и целое число K (> 0). Записать в текстовый файл с именем name K символов «*». Использовать алгоритм fill_n.
8	Даны вещественные числа A, D и целое число N. Вывести N первых членов арифметической прогрессии с первым элементом A и разностью D. Использовать алгоритм generate_n.
9	Дана строка name и целое число N (1 ≤ N ≤ 26). Записать в текстовый файл с именем name N начальных прописных букв латинского алфавита. Использовать алгоритм generate_n.

Задание 5. Алгоритмы поиска.

1	Дан вектор V. Удалить второй из элементов вектора, равных нулю. Если нулевых элементов меньше двух, то вектор не изменять. Использовать два вызова алгоритма find и функцию-член erase.
---	---

2	Дан дек D. Удалить последний нулевой элемент дека. Если нулевых элементов нет, то дек не изменять. Использовать алгоритм find с обратными итераторами и функцию-член erase. Указание. Алгоритм find может возвращать обратный итератор, однако функция-член erase не позволяет его использовать для удаления элемента. Необходимо перейти от обратного итератора r к связанному с ним обычному итератору, используя функцию-член обратного итератора r.base(). При этом следует учитывать, что функция r.base() возвращает итератор, связанный с элементом, следующим за тем, с которым связан обратный итератор r. Поэтому в функции-члене erase следует указать одно из следующих выражений (предполагается, что r — это обратный итератор, который вернул алгоритм find, и этот итератор отличен от rend): -- r.base() или (++r).base().
3	Дан список L. Удалить первый и последний нулевой элемент списка. Если нулевых элементов нет, то список не изменять, если нулевой элемент всего один, то удалить только его. Использовать два вызова алгоритма find и два вызова функции-члена erase. Указание. Имеющуюся в классе list функцию-член remove в данном случае использовать нельзя, так как она удаляет все элементы с определенным значением. См. также указание к задаче 5.2.
4	Дан список L. Удалить все элементы списка, расположенные между первым и вторым отрицательным элементом (не включая сами отрицательные элементы). Если список не содержит отрицательных элементов, то не изменять его, если отрицательный элемент всего один, то удалить все элементы, следующие за этим отрицательным элементом. Использовать два вызова алгоритма find_if и один вызов функции-члена erase.
5	Дан список L, содержащий как отрицательные, так и положительные элементы. Вставить нулевой элемент после первого отрицательного элемента и перед последним положительным элементом. Использовать два вызова алгоритма find_if и два вызова функции-члена insert. Указание. По поводу использования алгоритма find_if с обратным итератором см. указание к задаче 5.2.
6	Даны вектор V и список L; вектор V имеет четное количество элементов. Продублировать последний элемент списка, совпадающий с каким-либо элементом из первой половины исходного вектора. Если список не содержит требуемых элементов, то не изменять его. Использовать алгоритм find_first_of и функцию-член insert для списка.
7	Дан вектор V с четным количеством элементов. Добавить нулевой элемент перед последним элементом в первой половине вектора, совпадающим с каким-либо элементом из второй половины этого же вектора. Если вектор не содержит требуемых элементов, то не изменять его. Использовать алгоритм find_first_of и функцию-член insert.
8	Дано целое число K (> 0) и вектор V, содержащий только нули и единицы. Удалить в векторе V последний набор из K подряд расположенных нулей (если в этом наборе имеется больше K нулей, то требуется удалить только последние K из них). Если вектор не содержит требуемого набора нулей, то не изменять его. Использовать алгоритм search_n и функцию-член erase, а также обратные итераторы.
9	Дано целое число K (> 0) и вектор V. Продублировать в векторе V первый набор из K подряд расположенных положительных чисел, вставив после этого набора его копию (если в наборе имеется больше K положительных чисел, то лишние числа во внимание не принимаются, и дублируется только начальная часть набора, содержащая K чисел). Если вектор не содержит требуемого набора чисел, то не изменять его. Использовать алгоритм search_n с параметром — функциональным объектом.

Задание 6. Базовые модифицирующие алгоритмы. Итераторы вставки.

1	Даны числа A и B и векторы V1 и V2, каждый из которых содержит не менее 10 элементов. Заполнить первые 5 элементов каждого вектора значениями A, а последние 5 — значениями B. При преобразовании вектора V1 использовать два вызова алгоритма fill, при преобразовании вектора V2 использовать два вызова алгоритма fill_n.
2	Даны числа A и B и векторы V1 и V2. Добавить в начало каждого вектора 5 элементов со значениями A, а в конец — 5 элементов со значениями B. При преобразовании вектора V1 использовать два вызова алгоритма fill_n с функциями inserter и back_inserter (данные функции возвращают итераторы вставки), при преобразовании вектора V2 использовать два вызова функции-члена insert. Замечание. Второй способ является более эффективным.

3	Дано число $N (> 0)$ и дек D , содержащий не менее $2N$ элементов. Заполнить первые N элементов дека последовательностью чисел 1, 2, ..., N , а последние N элементов дека — последовательностью $N, N-1, \dots, 2, 1$. Использовать два вызова алгоритма generate с одинаковым параметром — функциональным объектом, а также обратные итераторы.
4	Дано число $N (> 0)$ и дек D . Добавить в начало дека последовательность чисел 1, 2, ..., N , а в его конец — последовательность $N, N-1, \dots, 2, 1$. Использовать два вызова алгоритма generate_n с одинаковым параметром — функциональным объектом, а также итераторы вставки, возвращаемые функциями front_inserter и back_inserter.
5	Даны списки $L1$ и $L2$, каждый из которых содержит четное количество элементов. Поменять местами первую и вторую половину каждого списка (например, список 1, 2, 3, 4 должен быть преобразован следующим образом: 3, 4, 1, 2). Для первого списка использовать алгоритм swap_ranges, для второго — алгоритм rotate. Использовать также функцию advance
6	Дано число $K (0 < K < 10)$ и списки $L1$ и $L2$, каждый из которых содержит не менее 10 элементов. Выполнить для списка $L1$ циклический сдвиг элементов вправо на K позиций, а для списка $L2$ — циклический сдвиг влево на K позиций. Использовать алгоритм rotate и функцию advance.
7	Дано число $K (0 < K < 5)$ и список L , содержащий не менее 10 элементов. Набор из первых 5 элементов списка скопировать в конец списка, выполнив для этой копии циклический сдвиг на K позиций вправо, а набор из последних 5 элементов исходного списка скопировать в начало списка, выполнив для этой копии циклический сдвиг на K позиций влево. Использовать два вызова алгоритма rotate_copy и итераторы вставки, а также функцию advance.
8	Даны списки $L1$ и $L2$, у каждого из которых количество элементов делится на 4. В первом списке инвертировать (расположить в обратном порядке) первую половину элементов, во втором списке — вторую половину. Для первого списка использовать алгоритм swap_ranges и обратные итераторы, для второго — алгоритм reverse. Использовать также функцию advance.
9	Даны списки $L1$ и $L2$, каждый из которых содержит четное количество элементов. В каждом списке продублировать первую половину, добавив ее элементы в конец списка в обратном порядке. Для первого списка использовать алгоритм reverse_copy и итератор вставки, для второго — функцию-член insert и обратные итераторы. Использовать также функцию advance. Замечание. Второй способ является более эффективным.
10	Дан вектор V с четным количеством элементов. В первой половине исходного вектора заменить все отрицательные числа на -1 , а во второй — все положительные числа на 1. Использовать два вызова алгоритма replace_if с различными параметрами — функциональными объектами.
11	Дан список L с четным количеством элементов. Скопировать в конец списка все элементы, расположенные в его первой половине, заменив при этом отрицательные элементы на нули и расположив скопированные элементы в обратном порядке. Использовать алгоритм replace_copy_if, итератор вставки и обратные итераторы, а также функцию advance.
12	Дан дек D с четным количеством элементов. Решить для него задачу 6.11. Поскольку операция вставки делает все итераторы дека недействительными, для решения задачи использовать вспомогательный дек $D0$. Инициализировать дек $D0$ первой половиной элементов дека D , после чего применить алгоритм replace_copy_if, используя дек $D0$ как источник, а дек D как приемник данных.
13	Дан список L с четным количеством элементов. Скопировать в начало списка все положительные элементы, расположенные в его второй половине, сохранив для них исходный порядок следования. Использовать алгоритм remove_copy_if и итератор вставки, а также функцию advance.
14	Дан вектор V с четным количеством элементов. Решить для него задачу 6.13. Поскольку операция вставки в начало вектора делает все его итераторы недействительными, для решения задачи использовать вспомогательный вектор $V0$. Инициализировать вектор $V0$ второй половиной элементов вектора V , после чего применить алгоритм remove_copy_if, используя вектор $V0$ как источник, а вектор V как приемник данных.
15	Дан вектор V с четным количеством элементов. Удалить все нулевые элементы, расположенные во второй половине исходного вектора. Использовать алгоритм remove и функцию-член erase. Указание. Алгоритм remove и другие алгоритмы, связанные с удалением элементов, не удаляют требуемые элементы, а лишь перемещают их в конец указанного диапазона и возвращают итератор на начало диапазона с перемещенными элементами. Для фактического удаления элементов после выполнения алгоритма необходимо вызвать функцию-член erase.

Задание 7. Сортировка и слияние.

1	Дан вектор V с нечетным количеством элементов N (≥ 3). Определить значения трех средних элементов вектора после того, как вектор будет отсортирован (по возрастанию), и вывести их в порядке возрастания. Использовать алгоритмы <code>nth_element</code> (для нахождения центрального элемента), <code>max_element</code> (для нахождения элемента, предшествующего центральному) и <code>min_element</code> (для нахождения элемента, следующего за центральным).
2	Дан вектор V , содержащий не менее 3 элементов. Определить значения трех начальных элементов вектора после того, как вектор будет отсортирован (по возрастанию), и вывести их в порядке возрастания. Использовать один вызов алгоритма <code>partial_sort</code> и алгоритм <code>copy</code> для вывода требуемых элементов.
3	Дан вектор V , содержащий не менее 3 элементов. Определить значения трех конечных элементов вектора после того, как вектор будет отсортирован (по возрастанию) и вывести их в порядке убывания. Использовать один вызов алгоритма <code>partial_sort</code> с параметром — функциональным объектом <code>greater</code> и алгоритм <code>copy</code> для вывода требуемых элементов.
4	Дан вектор V с четным количеством элементов. Добавить в его конец первую половину элементов, которые содержал бы отсортированный по возрастанию вариант исходного вектора. Использовать функцию-член <code>insert</code> для увеличения количества элементов вектора на требуемую величину и алгоритм <code>partial_sort_copy</code> .
5	Дан список L . Определить количество четных и нечетных чисел в исходном списке (вначале вывести количество четных, затем количество нечетных чисел). Использовать алгоритм <code>partition</code> и два вызова функции <code>distance</code> для итераторов.
6	Дан список L . Перегруппировать элементы списка, расположив в нем вначале положительные, а затем неположительные элементы (порядок расположения элементов в каждой группе должен совпадать с исходным). Использовать алгоритм <code>stable_partition</code> .
7	Дан список L . Перегруппировать элементы списка, расположив в нем вначале отрицательные, затем нулевые, а затем положительные элементы (порядок расположения элементов в каждой группе должен совпадать с исходным). Использовать два вызова алгоритма <code>stable_partition</code> .
8	Дан список L с четным количеством элементов. Перегруппировать элементы списка, расположив в нем вначале все четные элементы из первой половины исходного списка, затем все нечетные элементы, а затем — все четные элементы из второй половины списка (порядок расположения элементов в каждой группе должен совпадать с исходным). Использовать два вызова алгоритма <code>stable_partition</code> .
9	Дан дек D . Решить для него задачу 7.7, используя один вызов алгоритма <code>stable_sort</code> с параметром — функциональным объектом.
10	Дан дек D , элементами которого являются английские слова. Отсортировать его элементы по возрастанию их длин, а элементы одинаковой длины — в алфавитном порядке. Использовать алгоритм <code>sort</code> для сортировки по алфавиту и затем алгоритм <code>stable_sort</code> с параметром — функциональным объектом для сортировки по длине. Выводить новое содержимое дека после применения каждого алгоритма.
11	Дан дек D , элементами которого являются английские слова. Отсортировать его элементы по убыванию их длин, а элементы одинаковой длины — в алфавитном порядке. Использовать единственный вызов алгоритма <code>sort</code> с параметром — функциональным объектом, включающим как сравнение строк, так и сравнение их длин.
12	Дан вектор V с четным количеством элементов. Известно, что первая половина вектора уже отсортирована по возрастанию. Отсортировать все элементы вектора по возрастанию, выполнив вначале сортировку его второй половины алгоритмом <code>sort</code> , а затем слияние обеих половин алгоритмом <code>inplace_merge</code> . Выводить новое содержимое вектора V после применения каждого алгоритма.
13	Дан вектор V , количество элементов которого делится на 3. Известно, что каждая треть вектора уже упорядочена по возрастанию. Решить для исходного вектора задачу 7.7, используя два вызова алгоритма <code>inplace_merge</code> с одним и тем же параметром — функциональным объектом.

Задание 8. Численные алгоритмы.

1	Дан список L. Получить вектор V вещественных чисел, содержащий значения среднего арифметического для всех пар соседних элементов исходного списка (количество элементов вектора V должно быть на 1 меньше количества элементов списка L). Например, для исходного списка 1, 3, 4, 6 полученный вектор должен содержать значения 2.0, 3.5, 5.0. Использовать алгоритм adjacent_difference с итератором вставки и функциональным объектом, а также функцию-член erase для вектора V.
2	Дан список L, элементами которого являются английские слова. Получить дек D со строковыми элементами, каждый из которых строится по паре соседних элементов исходного списка L следующим образом: последняя буква правого элемента пары приписывается справа к первой букве левого элемента пары. Количество элементов дека D должно быть на 1 меньше количества элементов списка L. Например, для исходного списка ABC, DEF, KLM, XYZ полученный дек должен содержать строки AF, DM, KZ. Использовать алгоритм adjacent_difference с итератором вставки и функциональным объектом, а также функцию-член erase для дека D.

Задание 9. Множества.

1	Дан вектор V0, целое число N (> 0) и набор векторов V1, ..., VN. Известно, что размер вектора V0 не превосходит размера любого из векторов V1, ..., VN. Найти количество векторов VI, I = 1, ..., N, в которых содержатся все элементы вектора V0 (без учета их повторений). Использовать алгоритм includes, применяя его в цикле к двум множествам, одно из которых создано на основе вектора V0, а другое на очередной итерации содержит элементы очередного из векторов VI, I = 1, ..., N.
2	Дан вектор V0, целое число N (> 0) и набор векторов V1, ..., VN. Известно, что размер вектора V0 не превосходит размера любого из векторов V1, ..., VN. Найти количество векторов VI, I = 1, ..., N, в которых содержатся все элементы вектора V0 (с учетом их повторений). Использовать алгоритм includes, применяя его в цикле к двум мультимножествам (контейнерам типа multiset), одно из которых создано на основе вектора V0, а другое на очередной итерации содержит элементы очередного из векторов VI, I = 1, ..., N.

Задание 10. Отображения.

1	Дан вектор V. Определить количество повторений каждого числа в векторе V и вывести все различные элементы вектора V вместе с количеством их повторений (в порядке возрастания значений элементов); количество повторений выводить сразу после значения соответствующего элемента. Использовать вспомогательное отображение M (класс map), ключами которого являются различные элементы вектора V, а значениями — количество повторений этих элементов. При заполнении отображения M не использовать условные конструкции (достаточно операций индексирования [] и инкремента). Элементы вектора V (при заполнении отображения M) и элементы отображения M (при выводе полученных результатов) перебирать в цикле с параметром-итератором соответствующего контейнера.
2	Дан вектор V, элементами которого являются английские слова, набранные заглавными буквами. Определить суммарную длину слов, начинающихся с одной и той же буквы, и вывести все различные буквы, с которых начинаются элементы вектора V, вместе с суммарной длиной этих элементов (в алфавитном порядке букв); длину выводить сразу после соответствующей буквы. Использовать вспомогательное отображение M, ключами которого являются начальные буквы элементов вектора V, а значениями — суммарная длина этих элементов. При заполнении отображения M не использовать условные конструкции (достаточно операций индексирования [], инкремента и функции-члена size для строк). Элементы вектора V (при заполнении отображения M) и элементы отображения M (при выводе полученных результатов) перебирать в цикле с параметром-итератором соответствующего контейнера.

3	Дан вектор V. Выполнить группировку элементов вектора V, используя в качестве ключа группировки последнюю (т. е. правую) цифру элемента: в одну группу должны входить все элементы вектора V, оканчивающиеся на одну и ту же цифру (сгруппированные элементы должны располагаться в том же порядке, в котором они располагались в исходном векторе). Представить результат группировки в виде мультиотображения M (класса <code>multimap</code>), ключами которого являются ключи группировки, т. е. последние цифры элементов вектора V, а значениями — элементы вектора, оканчивающиеся на соответствующую цифру (таким образом, отображение M должно иметь тип <code>multimap<int, int></code>). Вывести полученное отображение (для каждого элемента отображения M вначале выводить ключ, а затем связанный с ним элемент вектора V; ключи могут повторяться). Для перебора элементов контейнеров использовать циклы с параметрами-итераторами.
4	Дан вектор V, элементами которого являются английские слова, набранные заглавными буквами. Выполнить группировку элементов вектора V, используя в качестве ключа группировки последнюю букву элемента: в одну группу должны входить все элементы вектора V, оканчивающиеся одной и той же буквой (сгруппированные элементы должны располагаться в порядке, обратном порядку их расположения в исходном векторе). Представить результат группировки в виде мультиотображения M, ключами которого являются ключи группировки, т. е. последние буквы элементов вектора V, а значениями — элементы вектора, оканчивающиеся на соответствующую букву (таким образом, отображение M должно иметь тип <code>multimap<char, string></code>). Вывести полученное отображение (для каждого элемента отображения M вначале выводить ключ, а затем связанный с ним элемент вектора V; ключи могут повторяться). Для перебора элементов контейнеров использовать цикл <code>for</code> по элементам контейнера.